

Feladat: Támadás

Készítsünk egy *WPF* grafikus felületű, *MVVM* architektúrájú alkalmazást *C#* nyelven a következő két személyes játékra. A játékot egy $n \times n$ -es táblán játsszák, kezdetben 4-4 figurával, amelyek a két ellentétes sarokban helyezkednek el, az ábrán látható módon. A figurákkal nyolc szomszédos irányba lehet lépni (egy mezőt), illetve átlósan lehet ütni (egy mezőt). A játékosok egyenként lépegetnek, azonban a figurákkal csak a meghatározott sorrendben léphetnek (azaz első körben az 1-es, második körben a 2-es, ötödik körben ismét az 1-es bábuval léphetnek, és így tovább). A játék célja, hogy a játékos egy figurája eljusson a szemközti sarokba.

				3	4
				2	1
1	2				
4	3				

Részfeladatok:

1. A program jelenítse meg a játéktáblát, amelynek mérete legyen 6×6 -os. A táblán legyenek láthatóak mindkét játékos bábuja a kezdőpozícióban.
2. A játékosoknak legyen lehetősége egymást váltva lépegetni a soron következő bábuval (lépéskor elég a célmezőt kijelölni, mivel egyértelmű, melyik bábuval lép a játékos). A program csak szabályos lépéseket tegyen lehetővé, illetve folyamatosan jelezze a képernyőn, melyik játékos következik, hányas számú figurával.

GameModel	figures.Add(new Figure)
<pre> namespace Attack.Model { public class FigureChangedEventArgs : EventArgs { public required FigureData Data { get; init; } public required Position NewPosition { get; init; } public required Position OldPosition { get; init; } } public class GameModel { private List<Figure> _figures; public int Size { get; init; } public event EventHandler<FigureChangedEventArgs>? FigureChanged; public event EventHandler<Player>? GameWon; public FigureData? GetByPosition(int row, int col) { // Megkeressük a listában, van-e bábu az adott pozíción. return _figures.FirstOrDefault(f => f.Position.Row == row && f.Position.Col == col)?.Data; } // Az éppen soron következő bábu mindig a lista legelső eleme (0. index). public FigureData CurrentFigure => _figures[0].Data; public GameModel(int size) { Size = size; _figures = new List<Figure>(); // --- BÁBUK FELVÉTELE A LISTÁBA --- // A hozzáadás sorrendje határozza meg, hogy ki mikor lép! // 1. bábu (Piros 1-es) _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 1 }, Position = new Position { Row = Size - 2, Col = 0 } }); // 2. bábu (Kék 1-es) _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 1 }, Position = new Position { Row = 1, Col = Size - 1 } }); // ... a többi bábu ... _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 2 }, Position = new Position { Row = Size - 2, Col = 1 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 2 }, Position = new Position { Row = 1, Col = Size - 2 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 3 }, Position = new Position { Row = Size - 1, Col = 1 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 3 }, Position = new Position { Row = 0, Col = Size - 2 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 4 }, Position = new Position { Row = Size - 1, Col = 0 } }); } } } </pre>	<pre> Data = new FigureData { Owner = Player.Red, Id = 1 }, Position = new Position { Row = Size - 2, Col = 0 } }); // 2. bábu (Kék 1-es) _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 1 }, Position = new Position { Row = 1, Col = Size - 1 } }); // ... a többi bábu ... _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 2 }, Position = new Position { Row = Size - 2, Col = 1 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 2 }, Position = new Position { Row = 1, Col = Size - 2 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 3 }, Position = new Position { Row = Size - 1, Col = 1 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Blue, Id = 3 }, Position = new Position { Row = 0, Col = Size - 2 } }); _figures.Add(new Figure { Data = new FigureData { Owner = Player.Red, Id = 4 }, Position = new Position { Row = Size - 1, Col = 0 } }); </pre>

```

{
    Data = new FigureData { Owner = Player.Blue, Id = 4 },
    Position = new Position { Row = 0, Col = Size - 1 }
});
}
public void Move(int row, int col)
{
    // 1. Pályán kívüli kattintás ellenőrzése
    if (row < 0 || row >= Size || col < 0 || col >= Size)
        return;

    // Mivel mindig a lista elején lévő bábu jön, lekérjük a 0. elemet.
    var currentFigure = _figures[0];

    // Kiszámoljuk, mennyit mozdult a bábu
    var diffRow = row - currentFigure.Position.Row;
    var diffCol = col - currentFigure.Position.Col;

    // 2. Szabálytalan lépés szűrése:
    // Túl messze lépett (>1) VAGY helyben maradt (0).
    if (Math.Abs(diffRow) > 1 || Math.Abs(diffCol) > 1 || Math.Abs(diffRow)
+ Math.Abs(diffCol) == 0)
        return;

    // 3. Van-e ott saját bábu? (Arra nem léphetünk)
    if (_figures.Any(f => f.Position.Row == row && f.Position.Col == col
&&
f.Data.Owner == currentFigure.Data.Owner))
        return;

    // Megnézzük, van-e ott ellenfél (ütés lehetősége)
    var collisionFigure = _figures.FirstOrDefault(f => f.Position.Row ==
row && f.Position.Col == col);
    if (collisionFigure != null)
    {
        // Ha saját, akkor return (bár ezt fentebb már szűrtük)
        if (collisionFigure.Data.Owner == currentFigure.Data.Owner)
            return;

        // ÜTÉS SZABÁLY: Csak átlósan szabad ütni!
        // Ha nem átlós (vagyis nem mindkét koordináta változott 1-gyel),
        akkor érvénytelen.
        if (Math.Abs(diffRow) != 1 || Math.Abs(diffCol) != 1)
            return;
    }

    // --- LÉPÉS VÉGREHAJTÁSA ---
    var oldPosition = currentFigure.Position;
    currentFigure.Position = new Position { Row = row, Col = col };

    // Ha volt ott ellenfél, azt töröljük a listából (LEÜTÉS)
    if (collisionFigure != null)
    {
        _figures.Remove(collisionFigure);
    }

    // --- FORGATÁS (ROTATION) ---
    // A lépés végén az aktuális bábu a sor végére kerül.
    // 1. Kivesszük az elejéről (0 volt a 0.)
    _figures.RemoveAt(0);
    // 2. Betesszük a lista végére
    _figures.Add(currentFigure);

    // Jelezzük a felületnek (ViewModel), hogy változás történt
    FigureChanged?.Invoke(this, new FigureChangedEventArgs
{
    Data = currentFigure.Data,
    NewPosition = currentFigure.Position,
    OldPosition = oldPosition
});

    // --- GYŐZELEM ELLENŐRZÉSE ---
    // Piros célja: (0, Size-1) sarok
    if (currentFigure.Data.Owner == Player.Red &&
currentFigure.Position.Row == 0 && currentFigure.Position.Col ==
Size - 1)
    {
        GameWon?.Invoke(this, Player.Red);
    }
    // Kék célja: (Size-1, 0) sarok
    else if (currentFigure.Data.Owner == Player.Blue &&
currentFigure.Position.Row == Size - 1 &&
currentFigure.Position.Col == 0)
    {
        GameWon?.Invoke(this, Player.Blue);
    }
}

```

```

Figure
public struct Position
{
    public required int Row { get; init; }
    public required int Col { get; init; }
}

public enum Player { Red, Blue }

public struct FigureData
{
    public required Player Owner { get; init; }
    public required int Id { get; init; }
}

public class Figure
{
    public FigureData Data { get; init; }
    public Position Position { get; set; }
}

CellViewModel
[ObservableProperty]
private Player? _owner;

[ObservableProperty]
private string _text = string.Empty;

public Position Position { get; init; }

public RelayCommand? ClickCommand { get; set; }

```

GameViewModel

```
public partial class GameViewModel : ObservableObject
{
    private GameModel _model;

    public ObservableCollection<CellViewModel> Cells { get; set; }

    public int Size => _model.Size;

    public FigureData CurrentFigure => _model.CurrentFigure;

    public GameViewModel(GameModel model)
    {
        _model = model;
        _model.FigureChanged += OnFigureChanged;

        Cells = new ObservableCollection<CellViewModel>();
        for (int row = 0; row < _model.Size; row++)
        {
            for (int col = 0; col < _model.Size; col++)
            {
                var data = _model.GetByPosition(row, col);
                // Létrehozzuk a mező ViewModel-jét (ez egyetlen gomb
adatait tárolja)
                var cell = new CellViewModel()
                {
                    Position = new Position { Row = row, Col = col }

                    if (data != null)
                    {
                        cell.Owner = data.Value.Owner;
                        cell.Text = data.Value.Id.ToString();
                    }

                    //var rowCopy = row;
                    //var colCopy = col;

                    cell.ClickCommand = new RelayCommand(() =>
                    {
                        _model.Move(cell.Position.Row, cell.Position.Col);
                    });

                    Cells.Add(cell);
                }
            }
        }

        private void OnFigureChanged(object? sender,
FigureChangedEventArgs e)
        {
            // 1. A RÉGI helyről levesszük a bábút.
            // Matek: Mivel a lista egydimenziós (0..35), de a tábla
kétdimenziós (sor, oszlop),
            // így számoljuk ki az indexet: (sor * szélesség + oszlop)
            var fromCell = Cells[e.OldPosition.Row * _model.Size +
e.OldPosition.Col];
            fromCell.Owner = null;
            fromCell.Text = string.Empty;

            var toCell = Cells[e.NewPosition.Row * _model.Size +
e.NewPosition.Col];
            toCell.Owner = e.Data.Owner;
            toCell.Text = e.Data.Id.ToString();

            OnPropertyChanged(nameof(CurrentFigure));
        }
    }
}
```

MainWindow

```
<Grid>
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="*" />
    <ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>

<ItemsControl ItemsSource="{Binding Cells}" Grid.Column="0">
    <ItemsControl.ItemsPanel>
        <ItemsPanelTemplate>
            <UniformGrid Columns="{Binding Size}" Rows="{Binding
Size}" />
        </ItemsPanelTemplate>
    </ItemsControl.ItemsPanel>
    <ItemsControl.ItemTemplate>
        <DataTemplate>
            <Button Content="{Binding Text}" Command="{Binding
ClickCommand}" FontSize="30">
                <Button.Style>
                    <Style TargetType="Button">
                        <Setter Property="Background" Value="White"/>
                        <Style.Triggers>
                            <DataTrigger Binding="{Binding Owner}"
Value="Red">
                                <Setter Property="Background" Value="Red"/>
                            </DataTrigger>
                            <DataTrigger Binding="{Binding Owner}"
Value="Blue">
                                <Setter Property="Background" Value="Blue"/>
                            </DataTrigger>
                        </Style.Triggers>
                    </Style>
                </Button.Style>
            </Button>
        </DataTemplate>
    </ItemsControl.ItemTemplate>
</ItemsControl>

<Label Grid.Column="1" Content="{Binding CurrentFigure.Id}"
Width="50" Height="50"
HorizontalContentAlignment="Center"
VerticalAlignment="Center"
FontSize="30">
    <Label.Style>
        <Style TargetType="Label">
            <Style.Triggers>
                <DataTrigger Binding="{Binding CurrentFigure.Owner}"
Value="Red">
                    <Setter Property="Background" Value="Red"/>
                </DataTrigger>
                <DataTrigger Binding="{Binding CurrentFigure.Owner}"
Value="Blue">
                    <Setter Property="Background" Value="Blue"/>
                </DataTrigger>
            </Style.Triggers>
        </Style>
    </Label.Style>
</Label>
</Grid>

App
public App()
{
    Startup += App_Startup;
}

private void App_Startup(object sender, StartupEventArgs e)
{
    GameModel model = new GameModel(6);
    model.GameWon += Model_GameWon;

    GameViewModel vm = new GameViewModel(model);
    MainWindow window = new MainWindow
    {
        DataContext = vm
    };
    window.Show();
}

private void Model_GameWon(object? sender, Player e)
{
    MessageBox.Show($"{e} player has won!", "Game Over",
MessageBoxButton.OK, MessageBoxImage.Information);
}
```

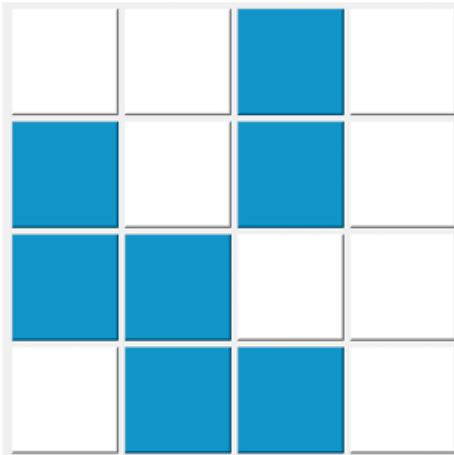
Feladat: BlockDocu

Valósítsunk meg *Windows Forms* grafikus felületű alkalmazásként, *MV* architektúrával, *C#* nyelven egy olyan játékot, amely során egy 4x4-es táblában kell alakzatokat elhelyeznünk. Négyféle alakzat fordulhat elő: két darab 2 blokk méretű (vízszintesen, illetve függőlegesen), és két darab 3 blokk méretű (L alakú, és ennek tükörképe). Ezt a négy alakzatot kék színnel jelöltük. (A piros pont a bal felső sarkukat jelöli, amelynek az 1. részfeladatban lesz szerepe.)



Ezek közül a játékos minden körben véletlenszerűen kap egyet, és azt kell elhelyeznie a játéktáblán úgy, hogy a játékos az alakzat bal felső blokkjának (ezt jelöltük piros ponttal) helyét jelöli meg a táblán. Ügyelni kell azonban arra, hogy az alakzat ne lógjon ki a tábláról, és ne fedjen le másikat, már fenn levő alakzatot. Ha egy sor vagy oszlop megtelik, annak mezői újra üressé válnak.

Minta a játék egy állapotára:



Részfeladatok:

1. Az alkalmazást kétrétegű modell-nézet architektúrában valósítsuk meg: a modell és a nézet legyen külön osztályban megvalósítva. A program mutassa meg az üres (fehér blokkokból álló) 4x4-es táblát, és külön egy 2x2-es keretben a véletlen kiválasztott alakzatot. A felhasználónak el kell tudnia helyezni ezt az alakzatot a táblán úgy, hogy ne lógjon ki róla. Ha kilógna, az elhelyezés sikertelen, azt meg kell ismételni. Az alakzat elhelyezését a megfelelő blokkok kékre színezésével jelöljük.
2. Az elhelyezendő véletlenszerűen kiválasztott alakzatok sorban egymás után jelenjenek meg. Egy-egy alakzat elhelyezésénél ellenőrizzük azt is, hogy az csupa fehér színű blokkra kerül-e rá. Csak sikeres elhelyezés után mutassuk meg a következő alakzatot. A program ismerje fel azt is, ha egy sor vagy oszlop megtelik (csupa kék színű blokkból áll) és ekkor színezzé ennek a sornak vagy oszlopnak a blokkjait fehérre.

<pre> GameModel public class GameModel { // A 4x4-es játéktábla. true = foglalt (kék), false = üres (fehér). private bool[,] _board = new bool[4, 4]; private Random _random = new Random(); // Alakzat definíció: a bool tömb a relatív blokk pozíciókat adja meg. public class Shape { public bool[,] Layout { get; } public Shape(bool[,] layout) { Layout = layout; } } // [cite_start]// A lehetséges alakzatok listája (két 2-blokkos, két 3- blokkos L-alakú) [cite: 5, 6] private readonly List<Shape> _availableShapes = new List<Shape> { // 2-blokkos vízszintes new Shape(new bool[,] { { false, false }, { true, true } }), // 2-blokkos függőleges new Shape(new bool[,] { { true }, { true } }), // 3-blokkos L-alak (2x2 területen belül) new Shape(new bool[,] { { true, false }, { true, true } }), // 3-blokkos tükrözött L-alak (2x2 területen belül) new Shape(new bool[,] { { true, true }, { false, true } }) }; public Shape? CurrentShape { get; private set; } // Események a Nézet értesítésére public event EventHandler? GameBoardChanged; public event EventHandler? NewShapeAvailable; public GameModel() { // [cite_start]// Kezdekör generálunk egy alakzatot GenerateNextShape(); } // 1. és 2. részfeladat: Új alakzat generálása public void GenerateNextShape() { int index = _random.Next(_availableShapes.Count); CurrentShape = _availableShapes[index]; // Értesíti a nézetet, hogy mutassa meg az új alakzatot NewShapeAvailable?.Invoke(this, EventArgs.Empty); } // 1. és 2. részfeladat: Ellenőrzés: Fed-e le a tábláról? [cite: 8, 14, 15] public bool TryPlaceShape(int startRow, int startCol) { if (CurrentShape == null) return false; int shapeHeight = CurrentShape.Layout.GetLength(0); int shapeWidth = CurrentShape.Layout.GetLength(1); // [cite_start]// 1. Ellenőrzés: Kilóg-e a tábláról? [cite: 8, 14, 15] if (startRow + shapeHeight > 4 startCol + shapeWidth > 4) { return false; } // [cite_start]// 2. Ellenőrzés: Fed-e le már foglalt (kék) blokkot? [cite: 8, 21] for (int i = 0; i < shapeHeight; i++) { for (int j = 0; j < shapeWidth; j++) { // Ha az alakzat blokkja 'true' ÉS a táblán is foglalt (true), akkor ütközés van if (CurrentShape.Layout[i, j] && _board[startRow + i, startCol + j]) { return false; // Lefed egy kék blokkot } } } } </pre>	<pre> // [cite_start]// 3. Sikeres elhelyezés: Frissíti a táblát (kékre színezi a blokkokat) [cite: 15] for (int i = 0; i < shapeHeight; i++) { for (int j = 0; j < shapeWidth; j++) { if (CurrentShape.Layout[i, j]) { _board[startRow + i, startCol + j] = true; } } } // Sorok/oszlopok ellenőrzése és tisztítása (2. részfeladat) CheckAndClearRowsAndColumns(); // Értesíti a nézetet a tábla változásáról GameBoardChanged?.Invoke(this, EventArgs.Empty); // [cite_start]// Csak sikeres elhelyezés után generálódik a következő alakzat [cite: 22] GenerateNextShape(); return true; } // [cite_start]// 2. részfeladat: Sorok és oszlopok ellenőrzése és tisztítá sa [cite: 9, 22] private void CheckAndClearRowsAndColumns() { // Sorok ellenőrzése for (int r = 0; r < 4; r++) { bool rowFilled = true; for (int c = 0; c < 4; c++) { if (!_board[r, c]) { rowFilled = false; break; } } if (rowFilled) { for (int c = 0; c < 4; c++) { _board[r, c] = false; } } } // Oszlopok ellenőrzése for (int c = 0; c < 4; c++) { bool colFilled = true; for (int r = 0; r < 4; r++) { if (!_board[r, c]) { colFilled = false; break; } } if (colFilled) { for (int r = 0; r < 4; r++) { _board[r, c] = false; } } } } // Segédmetódus a nézetnek a tábla állapotának lekérdezéséhez public bool GetCellStatus(int row, int col) { return _board[row, col]; } </pre>
--	--

<pre> View public partial class GameView : Form { private readonly GameModel _model; // A Form Designer által létrehozott vezérlők: // BoardPanel: 4x4 TableLayoutPanel a fő játéktáblának. // NextShapePanel: 2x2 TableLayoutPanel a következő alakzatnak. private Control[,] _boardBlocks = new Control[4, 4]; private Control[,] _nextShapeBlocks = new Control[2, 2]; public GameView() { InitializeComponent(); // A Form Designer által létrehozott // metódus _model = new GameModel(); // A Modell eseménykezelőinek feliratkozása _model.GameBoardChanged += Model_GameBoardChanged; _model.NewShapeAvailable += Model_NewShapeAvailable; SetupBoard(); // A 4x4-es és 2x2-es panelek feltöltése vezérlőkkel // Kezdeti állapot megjelenítése (1. részfeladat) UpdateBoardDisplay(); UpdateNextShapeDisplay(); } // Segédmetódus a vezérlők inicializálására (Panel vagy Button) private void SetupBoard() { // 4x4 Tábla beállítása for (int r = 0; r < 4; r++) { for (int c = 0; c < 4; c++) { // Használjunk Panel vezérlőt a blokk jelölésére Panel block = new Panel(); block.Click += BoardBlock_Click!; //// Mentjük a koordinátákat a Tag tulajdonságba a kattintás // azonosításához //block.Tag = new Point(r, c); _boardBlocks[r, c] = block; BoardPanel.Controls.Add(block, c, r); } } // 2x2 Alakzat panel beállítása (mindig üres marad, csak a szín // változik) for (int r = 0; r < 2; r++) { for (int c = 0; c < 2; c++) { Panel block = new Panel { Dock = DockStyle.Fill, Margin = new Padding(1) }; _nextShapeBlocks[r, c] = block; NextShapePanel.Controls.Add(block, c, r); } } // A Modellből jövő esemény: A tábla frissítése (1. és 2. részfeladat) private void Model_GameBoardChanged(object? sender, EventArgs e) { UpdateBoardDisplay(); } // A Modellből jövő esemény: A következő alakzat frissítése (1. ré // szfeladat) private void Model_NewShapeAvailable(object? sender, EventArgs e) { UpdateNextShapeDisplay(); } } </pre>	<pre> // A felhasználói interakció (kattintás) - Ide kattintással jelöli meg a bal // felső sarkot private void BoardBlock_Click(object? sender, EventArgs e) { // Az eseményt kiváltó vezérlő lekérése Control? clickedBlock = sender as Control; if (clickedBlock != null && _model != null) { // 1. Megkeressük a TableLayoutPanel-ben lévő sor- és // oszlopindexet. // A GetRow és GetColumn statikus metódusok lekérdezik a // vezérlő helyét a Panelben. int row = BoardPanel.GetRow(clickedBlock); int col = BoardPanel.GetColumn(clickedBlock); // 2. Ellenőrizzük, hogy a koordináták érvényesek-e (pl. -1 // lehet, ha nem található) if (row >= 0 && col >= 0) { // 3. Logika hívása a modellben bool success = _model.TryPlaceShape(row, col); if (!success) { MessageBox.Show("Hiba! Az alakzat kilóg a tábláról, vagy fed egy másik blokkot. Próbáld újra!", "Sikertelen elhelyezés"); } } } } // A 4x4 tábla megjelenítésének frissítése private void UpdateBoardDisplay() { for (int r = 0; r < 4; r++) { for (int c = 0; c < 4; c++) { // [cite_start]// Ha a modellben foglalt (true), akkor kék, // különben fehér [cite: 6, 13] _boardBlocks[r, c].BackColor = _model.GetCellStatus(r, c) ? Color.Blue : Color.White; } } } // A 2x2 alakzat keret megjelenítésének frissítése (1. részfeladat) private void UpdateNextShapeDisplay() { // Először minden blokkot fehérre állítunk for (int r = 0; r < 2; r++) { for (int c = 0; c < 2; c++) { _nextShapeBlocks[r, c].BackColor = Color.White; } } // Csak az alakzat blokkjait színezzük kékre a 2x2-es keretben if (_model.CurrentShape != null) { int shapeHeight = _model.CurrentShape.Layout.GetLength(0); int shapeWidth = _model.CurrentShape.Layout.GetLength(1); for (int r = 0; r < shapeHeight; r++) { for (int c = 0; c < shapeWidth; c++) { if (_model.CurrentShape.Layout[r, c]) { // Feltételezzük, hogy az alakzat befér a 2x2-es keretbe. if (r < 2 && c < 2) { _nextShapeBlocks[r, c].BackColor = Color.Blue; } } } } } } </pre>
---	---

Feladat: Gerrymandering

Pirézia városállamában hamarosan választásokat rendeznek, azonban a Kormányzó elégedetlen a várható választási kimenetellel. Készítsünk egy olyan *Windows Forms* grafikus felületű, *MV* architektúrájú alkalmazást *C#* nyelven, amellyel támogatjuk a Kormányzót, hogy a választások előtt előnyösen alakíthassa át a választókörzetek határait. Pirézia egy 3 x 6-os téglalap alapterületű, 18 kerületből álló város, minden kerület várható választási preferenciája ismert és megtalálható a mellékelt ábrán. A választási körzetek 3-3, összefüggő kerületből állnak. A választási körzetek ügyes kialakításával kétharmados győzelemre segíthetjük a Kormányzót (lila szín) a lakosság körében kisebbségi támogatása (8-10) ellenére.

G	G	P	G	P	G
P	G	P	G	G	P
G	P	G	G	P	P

G	G	P	G	P	G
P	G	P	G	G	P
G	P	G	G	P	P

G = Green Party (Zöld Párt), P = Purple Party (Lila Párt)

Részfeladatok:

2. A program jelenítse meg Pirézia kerületi térképét, amelyen jelölje, hogy a kerületek a Zöld Pártot vagy a Lila Pártot (Kormányzó) támogatják-e? A felhasználó választhasson ki 3 egymással összefüggő kerületet, amely egy választási körzet lesz. (Átlósan nem számítanak összefüggőnek a kerületek.) Érvénytelen kijelölést ne engedélyezzen a program. Az első körzet kijelölése után további 3 kerület kiválasztásával kijelölhető a következő körzet, és így tovább. Tetszőlegesen választott megoldással, de a program egyértelműen jelenítse meg a táblán, hogy mely kerületek alkotnak 1-1 választási körzetet. (Nem feltétlenül szükséges az ábrán látható jelölési módot implementálni ehhez.)

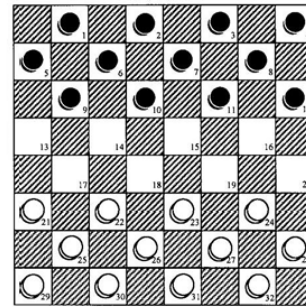
Legyen bármikor lehetőség előlről kezdeni a játékot!

<pre> GameModel public struct Position { public int Row { get; set; } public int Col { get; set; } } public class Kerulet { // Igaz, ha Lila párti kerület, hamis, ha zöld párti kerület public bool District { get; } public Position Position { get; } public int KorzetSzam { get; set; } = -1; // Alapértelmezett érték -1, ha nincs körzethez rendelve public Kerulet(bool district, Position position) { this.District = district; Position = position; } } public class GameModel { private Kerulet[,] _board; // Inicializálva a konstruktorban public List<Kerulet> AktualisKeruletek { get; set; } public List<List<Kerulet>> KivalasztottKorzetek { get; set; } public GameModel() { _board = new Kerulet[3, 6] { { new Kerulet(false, new Position{ Row = 0, Col = 0 }), new Kerulet(false, new Position{ Row = 0, Col = 1 }), new Kerulet(true, new Position{ Row = 0, Col = 2 }), new Kerulet(false, new Position{ Row = 0, Col = 3 }), new Kerulet(true, new Position{ Row = 0, Col = 4 }), new Kerulet(false, new Position{ Row = 0, Col = 5 }) }, { new Kerulet(true, new Position{ Row = 1, Col = 0 }), new Kerulet(false, new Position{ Row = 1, Col = 1 }), new Kerulet(true, new Position{ Row = 1, Col = 2 }), new Kerulet(false, new Position{ Row = 1, Col = 3 }), new Kerulet(false, new Position{ Row = 1, Col = 4 }), new Kerulet(true, new Position{ Row = 1, Col = 5 }) }, { new Kerulet(false, new Position{ Row = 2, Col = 0 }), new Kerulet(true, new Position{ Row = 2, Col = 1 }), new Kerulet(false, new Position{ Row = 2, Col = 2 }), new Kerulet(false, new Position{ Row = 2, Col = 3 }), new Kerulet(true, new Position{ Row = 2, Col = 4 }), new Kerulet(true, new Position{ Row = 2, Col = 5 }) } }; KivalasztottKorzetek = new List<List<Kerulet>>(); AktualisKeruletek = new List<Kerulet>(); // HIÁNYZOTT: Inicializálás } public event EventHandler? BoardChanged; public bool KorzetValasztas(int row, int col) { Kerulet selectedKerulet = _board[row, col]; // 1. Ellenőrzés: Már benne van az aktuális körzetben VAGY már lezárt körzet része if (AktualisKeruletek.Contains(selectedKerulet) KivalasztottKorzetek.Any(k => k.Contains(selectedKerulet))) { return false; } // 2. Ellenőrzés: Érvényes-e a választás? // Az első elemet (AktualisKeruletek.Count == 0) mindig elfogadjuk. bool isValidSelection = AktualisKeruletek.Count == 0; </pre>	<pre> if (AktualisKeruletek.Count > 0) { foreach (var currentKerulet in AktualisKeruletek) { // Ellenőrizzük, hogy a kiválasztott kerület szomszédos-e valamelyik már kiválasztott kerülettel int rowDiff = Math.Abs(currentKerulet.Position.Row - selectedKerulet.Position.Row); int colDiff = Math.Abs(currentKerulet.Position.Col - selectedKerulet.Position.Col); // Csak akkor szomszédos, ha a különbség 1 az egyik tengelyen ÉS 0 a másikon (nem átlósan) if ((rowDiff == 1 && colDiff == 0) (rowDiff == 0 && colDiff == 1)) { isValidSelection = true; break; // Elég egy szomszéd is a folytatáshoz } } } if (!isValidSelection) { return false; } // 3. Kiválasztás AktualisKeruletek.Add(selectedKerulet); // 4. Körzet lezárása if (AktualisKeruletek.Count == 3) { // Új: Meghatározzuk a következő körzet sorszámát int ujKorzetSzam = KivalasztottKorzetek.Count + 1; // Hozzárendeljük a sorszámat a 3 kerülethez foreach (var kerulet in AktualisKeruletek) { kerulet.KorzetSzam = ujKorzetSzam; } // Hozzáadjuk a lezárt körzetet a listához, majd töröljük az aktuális kerületeket KivalasztottKorzetek.Add(new List<Kerulet>(AktualisKeruletek)); AktualisKeruletek.Clear(); } BoardChanged?.Invoke(this, EventArgs.Empty); return true; } public Kerulet GetKeruletAt(int row, int col) { // Visszaadjuk a megadott pozíción lévő kerületet return _board[row, col]; } public void NewGame() { // Töröljük a kiválasztott körzeteket és az aktuális kerületeket KivalasztottKorzetek.Clear(); AktualisKeruletek.Clear(); // Végigmegyünk a táblán, és minden kerületről levesszük a sorsz ámot for (int row = 0; row < _board.GetLength(0); row++) { for (int col = 0; col < _board.GetLength(1); col++) { _board[row, col].KorzetSzam = -1; } } // Jelzés a nézetnek a frissítéshez BoardChanged?.Invoke(this, EventArgs.Empty); } </pre>
---	--

<pre> View private readonly GameModel _model; private Control[,] _boardControls = new Control[3, 6]; public GameView() { InitializeComponent(); _model = new GameModel(); _model.BoardChanged += Model_BoardChanged; // ÚJ: A tábla háttére fekete, ez adja meg a rács színét a gombok között Table.BackgroundColor = Color.Black; // Opcionális: A tábla körüli keret eltüntetése, ha zavaró Table.Padding = new Padding(2); Setuptable(); UpdateBoardDisplay(); Button newGameButton = new Button { Text = "Új játék", Location = new Point(0, 0), Size = new Size(90, 30) }; newGameButton.Click += NewGameButton_Click!; Controls.Add(newGameButton); } // Tábla létrehozása és inicializálása private void Setuptable() { // Töröljük a korábbi vezérlőket, ha lennének (biztonsági lépés) Table.Controls.Clear(); for (int row = 0; row < 3; row++) { for (int col = 0; col < 6; col++) { // Változás: Button használata Panel helyett Button block = new Button(); block.Click += TableBlock_Click; // Button alapstílus beállítása a szegélyek jobb kezeléséhez block.Dock = DockStyle.Fill; block.Margin = new Padding(2); block.FlatStyle = FlatStyle.Flat; // A szegély testreszabásához szükséges block.Font = new Font(block.Font, FontStyle.Bold); // P, G betűk kiemelése // Mentjük az azonosításhoz szükséges pozíciót block.Tag = new Position { Row = row, Col = col }; _boardControls[row, col] = block; Table.Controls.Add(block, col, row); } } } // Egy cella megjelenítésének frissítése private void UpdateBlockDisplay(int row, int col) { if (!(_boardControls[row, col] is Button block)) return; Kerulet kerulet = _model.GetKeruletAt(row, col); // 1. Szín beállítása if (kerulet.District) { block.BackgroundColor = Color.Purple; block.ForeColor = Color.White; } else { block.BackgroundColor = Color.LightGreen; block.ForeColor = Color.Black; } } </pre>	<pre> // 2. Szöveg beállítása (Párt + Körzet sorszám) string partyAbbr = kerulet.District ? "P" : "G"; string keruletText; if (kerulet.KorzetSzam > 0) { // Már lezárt körzet része keruletText = \$"{partyAbbr}-{kerulet.KorzetSzam}"; block.FlatStyleAppearance.BorderSize = 1; block.FlatStyleAppearance.BorderColor = Color.Black; } else if (_model.AktualisKeruletek.Contains(kerulet)) { // Éppen kijelölés alatt áll (aktuális kijelölés sárga háttérrel) keruletText = partyAbbr; block.BackgroundColor = Color.Yellow; block.ForeColor = Color.Black; // Eltüntetjük a keretet block.FlatStyleAppearance.BorderSize = 0; } else { // Nincs kijelölve: csak a párt betűje látszik keruletText = partyAbbr; // Eltüntetjük a keretet block.FlatStyleAppearance.BorderSize = 0; } block.Text = keruletText; } // Teljes tábla frissítése private void UpdateBoardDisplay() { for (int row = 0; row < 3; row++) { for (int col = 0; col < 6; col++) { UpdateBlockDisplay(row, col); } } } // Eseménykezelő egy tábla cella kattintására private void TableBlock_Click(object? sender, EventArgs e) { if (sender is Button clickedButton && clickedButton.Tag is Position position) { _model.KorzetValasztas(position.Row, position.Col); } } // Eseménykezelő a modell tábla változására private void Model_BoardChanged(object? sender, EventArgs e) { UpdateBoardDisplay(); } // Eseménykezelő az "Új játék" gombra private void NewGameButton_Click(object? sender, EventArgs e) { _model.NewGame(); UpdateBoardDisplay(); } </pre>
--	--

Feladat: Dáma

Készítsünk egy *Windows Presentation Foundation (WPF)* grafikus felületű, *MVVM* architektúrájú alkalmazást *C#* nyelven, amellyel a klasszikus Dáma játékot lehet játszani (egyszerűsített szabályokkal). A Dáma játékban két játékos (most fehér és fekete) játszanak egymás ellen, mindkettő három sor figurával (lásd ábra). A figurák egy mezőt léphetnek előre átlósan, vagy átugorhatnak egy ellenséges figurát (szintén átlósan), ha az ugrás irányában a figura mögött nincsen egy másik figura. Ezt hívják „ütésnek”, és ekkor a leütött figurát leszedjük a játéktérrel.



Részfeladatok:

2. A program jelenítsen meg egy 8 x 8-as játéktáblát és helyezze rá az ábrán látható figurákat (ezt lehet szöveggel, színnel vagy képpel is jelezni). Legyen lehetőség felváltva lépésre (az ütés is lépésnek számít) és jelezzük különböző hibaüzenetekkel, ha a felhasználó egy nem szabályos lépést szeretne elvégezni. Ha az egyik félnek nincs több figurája, vagy nincs több szabályos lépése, akkor vesztett. Ekkor legyen vége a játéknak és ezt felugró ablakkal jelezzük.

<pre> Figure public enum FigureData { Empty, White, Black } public struct Position { public int Row { get; set; } public int Col { get; set; } public Position(int row, int col) { Row = row; Col = col; } } public class Figure { public FigureData Figuredata { get; set; } public Position CurrentPosition { get; set; } public Figure(FigureData isWhite, Position position) { Figuredata = isWhite; CurrentPosition = position; } } </pre>	<pre> GameModel : Event public class FigureChangedEventArgs : EventArgs { public Position Position { get; } public FigureData FigureData { get; } public FigureChangedEventArgs(Position pos, FigureData data) { Position = pos; FigureData = data; } } public class PlayerTurnChangedEventArgs : EventArgs { public FigureData CurrentPlayer { get; } public PlayerTurnChangedEventArgs(FigureData player) => CurrentPlayer = player; } public class GameOverEventArgs : EventArgs { public FigureData Winner { get; } public GameOverEventArgs(FigureData winner) => Winner = winner; } public class InvalidActionEventArgs : EventArgs { public string Message { get; } public InvalidActionEventArgs(string msg) => Message = msg; } </pre>
---	---

<pre> GameModel public class GameModel { private List<Figure> _figures; public FigureData CurrentPlayer { get; set; } = FigureData.White; public Position? SelectedPosition { get; set; } = null; public event EventHandler<FigureChangedEventArgs>? FiguresChanged; public event EventHandler<PlayerTurnChangedEventArgs>? PlayerTurnChanged; public event EventHandler<GameOverEventArgs>? GameOver; public event EventHandler<InvalidActionEventArgs>? InvalidAction; public GameModel() { _figures = new List<Figure>(); InitializeFigures(); } private void InitializeFigures() { // Fekete (Fent, 0-2 sor) for (int row = 0; row < 3; row++) { for (int col = 0; col < 8; col++) { if ((row + col) % 2 == 1) _figures.Add(new Figure(FigureData.Black, new Position(row, col))); } } // Fehér (Lent, 5-7 sor) for (int row = 5; row < 8; row++) { for (int col = 0; col < 8; col++) { if ((row + col) % 2 == 1) _figures.Add(new Figure(FigureData.White, new Position(row, col))); } } } public List<Figure> GetFigures() => _figures; public void MakeMove(int row, int col) { var clickedPos = new Position(row, col); var clickedFigure = _figures.FirstOrDefault(f => f.CurrentPosition.Row == row && f.CurrentPosition.Col == col); // 1. Eset: Saját bábura kattintunk -> Kijelölés if (clickedFigure != null && clickedFigure.Figuredata == CurrentPlayer) { SelectedPosition = clickedPos; // Itt nem küldünk eseményt, csak megjegyezzük, hogy melyikkel akar lépni return; } // 2. Eset: Üres helyre kattintunk és VAN kijelölt bábu -> Lépés kísérlet if (SelectedPosition != null && clickedFigure == null) { var from = SelectedPosition.Value; if (IsValidMove(from, clickedPos)) { ExecuteMove(from, clickedPos); SelectedPosition = null; // Lépés után töröljük a kijelölést } } else { // Ha nincs kijelölve semmi, és üresre kattint, vagy ellenségre if (SelectedPosition == null) { InvalidAction?.Invoke(this, new InvalidActionEventArgs("Előbb jelölj ki egy saját bábút!")); } else { InvalidAction?.Invoke(this, new InvalidActionEventArgs("Ide nem léphetsz!")); } } private void ExecuteMove(Position from, Position to) { var figure = _figures.First(f => f.CurrentPosition.Row == from.Row && f.CurrentPosition.Col == from.Col); </pre>	<pre> // Ha ütés (távolság 2) if (Math.Abs(to.Row - from.Row) == 2) { int midRow = (from.Row + to.Row) / 2; int midCol = (from.Col + to.Col) / 2; var captured = _figures.FirstOrDefault(f => f.CurrentPosition.Row == midRow && f.CurrentPosition.Col == midCol); if (captured != null) { _figures.Remove(captured); FiguresChanged?.Invoke(this, new FigureChangedEventArgs(new Position(midRow, midCol), FigureData.Empty)); } } // Mozgatás figure.CurrentPosition = to; FiguresChanged?.Invoke(this, new FigureChangedEventArgs(from, FigureData.Empty)); FiguresChanged?.Invoke(this, new FigureChangedEventArgs(to, figure.Figuredata)); // Játékos váltás CurrentPlayer = CurrentPlayer == FigureData.White ? FigureData.Black : FigureData.White; PlayerTurnChanged?.Invoke(this, new PlayerTurnChangedEventArgs(CurrentPlayer)); CheckWin(); } private bool IsValidMove(Position from, Position to) { int rowDiff = to.Row - from.Row; int colDiff = Math.Abs(to.Col - from.Col); // Fehér (Lentről felfelé): Row csökken. rowDiff legyen negatív. if (CurrentPlayer == FigureData.White && rowDiff > 0) { InvalidAction?.Invoke(this, new InvalidActionEventArgs("Fehérrel csak előre (felfelé) léphetsz!")); return false; } // Fekete (Fentről lefelé): Row nő. rowDiff legyen pozitív. if (CurrentPlayer == FigureData.Black && rowDiff < 0) { InvalidAction?.Invoke(this, new InvalidActionEventArgs("Feketével csak előre (lefelé) léphetsz!")); return false; } // Sima lépés (1 átlós) if (Math.Abs(rowDiff) == 1 && colDiff == 1) return true; // Ütés (2 átlós) if (Math.Abs(rowDiff) == 2 && colDiff == 2) { int midRow = (from.Row + to.Row) / 2; int midCol = (from.Col + to.Col) / 2; var midFigure = _figures.FirstOrDefault(f => f.CurrentPosition.Row == midRow && f.CurrentPosition.Col == midCol); // Csak akkor ugorhat, ha van ott ellenséges bábu if (midFigure != null && midFigure.Figuredata != CurrentPlayer) return true; } InvalidAction?.Invoke(this, new InvalidActionEventArgs("Szabálytalan lépés!")); return false; } private void CheckWin() { // Egyszerűsített ellenőrzés: ha elfogytak a bábuk if (!_figures.Any(f => f.Figuredata == FigureData.White)) GameOver?.Invoke(this, new GameOverEventArgs(FigureData.Black)); else if (!_figures.Any(f => f.Figuredata == FigureData.Black)) GameOver?.Invoke(this, new GameOverEventArgs(FigureData.White)); } </pre>
--	---

FieldVM

```
private FigureData _figureData;
private string _text = string.Empty;

public FigureData FigureData
{
    get => _figureData;
    set
    {
        if (_figureData != value)
        {
            _figureData = value;
            OnPropertyChanged(); // Ez hiányzott! Ez frissíti a színt.
        }
    }
}

public string Text
{
    get => _text;
    set
    {
        if (_text != value)
        {
            _text = value;
            OnPropertyChanged(); // Ez frissíti a szöveget.
        }
    }
}

public Position Position { get; set; }
public DelegateCommand? ClickCommand { get; set; }
```

GameViewModel

```
private GameModel _model;
public ObservableCollection<FieldVM> Fields { get; set; }

// Események, amikre az App.xaml.cs feliratkozhat
public event EventHandler<string>? InvalidMove;
public event EventHandler<string>? GameFinished;

public GameViewModel(GameModel model)
{
    _model = model;
    _model.FiguresChanged += OnFiguresChanged;

    // Modell események átvezetése
    _model.InvalidAction += (s, e) => InvalidMove?.Invoke(this,
e.Message);
    _model.GameOver += (s, e) => GameFinished?.Invoke(this, (e.Winner
== FigureData.White ? "Fehér" : "Fekete") + " nyert!");

    Fields = new ObservableCollection<FieldVM>();

    // Tábla feltöltése (8x8)
    for (int row = 0; row < 8; row++)
    {
        for (int col = 0; col < 8; col++)
        {
            // !!! FONTOS JAVÍTÁS !!!
            // Létrehozunk egy másolatot a változóról a cikluson belül.
            // Ha ezt nem tesszük meg, a gomb mindig a (8,8)-as
            koordinátával próbálna lépni.
            int r = row;
            int c = col;

            var field = new FieldVM
            {
                Position = new Position(r, c),
                // Itt a másolatot (r, c) használjuk a parancshoz!
                ClickCommand = new DelegateCommand(param =>
                _model.MakeMove(r, c))
            };
            Fields.Add(field);
        }
    }
    RefreshTable(); // Kezdőállapot betöltése
}

private void OnFiguresChanged(object? sender,
FigureChangedEventArgs e)
{
    var field = Fields.FirstOrDefault(f => f.Position.Row ==
e.Position.Row && f.Position.Col == e.Position.Col);
    if (field != null) UpdateField(field, e.FigureData);
}

private void RefreshTable()
{
    foreach (var field in Fields)
    {
        var figure = _model.GetFigures().FirstOrDefault(f =>
f.CurrentPosition.Row == field.Position.Row && f.CurrentPosition.Col
== field.Position.Col);
        UpdateField(field, figure?.FigureData ?? FigureData.Empty);
    }
}

private void UpdateField(FieldVM field, FigureData data)
{
    field.FigureData = data;
    if (data == FigureData.White) field.Text = "O";
    else if (data == FigureData.Black) field.Text = "X";
    else field.Text = "";
}
```

Feladat: Invázió

A gonosz megszállók el akarják foglalni a várunkat, aminek a fala a pálya egyik szélén húzódik (oldaltól oldalig). Az áttellenes oldalról érkeznek az ellenségek, akik folyamatosan egyenes úton tartanak a várfalhoz. Azon ellenségek, akik elérik a várfalat, kiesnek a játékból, viszont a vár veszít egy életet. Ha a várnak mindhárom élete elfogy, akkor a játékos veszített.

A védelmet katonák biztosítják. Alapból csak két katona van, de minden harmadik megölt ellenségért cserébe lehet vásárolni egyet. A katonákat a pályára lehet helyezni, de felvenni nem lehet őket. Viselkedésük a következő:

- Ha katona ellenféllel ütközik, akkor mindketten meghalnak.
- Ha katona közelében elhalad egy ellenfél, de nem ütköznék össze, akkor csak az ellenfél hal meg.
- A katonák sosem mozdulnak el a posztjukról.

Készítsünk *Avalonia UI* keretrendszerben, *MVVM* architektúrájú mobilalkalmazást *C#* nyelven legalább egy támogatott platformra, amivel a fentebb vázolt játékot lehet játszani!

Részfeladatok:

1. Valósítsunk meg egy fix méretű pályát vezérlőkkel vagy elemi grafikával. Látszódjon a vár élete, az eltelt idő, és hogy hány katonát lehet még lehelyezni.
2. Időnként véletlen mennyiségű ellenfelek bukkannak fel véletlen pozíciókban a várral ellentétes oldalon, és egyenesen a vár felé haladjanak valós időben.

<pre>GameModel // Mezőtípusok a megjelenítéshez public enum Fieldtype { Empty, Wall, Enemy, Soldier } public class InvasionGameModel { #region Konstansok public const int Rows = 12; // Pálya magassága public const int Columns = 6; // Pálya szélessége private const int GameTickSpeed = 600; // Játék sebessége (ms) #endregion #region Mezők private readonly ITimer _timer; private readonly Random _random; // Játékállapot private int _castleLives; private int _enemiesKilled; private int _deployedSoldiers; private int _timeElapsed; // Entitások pozíciói private List<Tuple<int, int>> _enemies; private List<Tuple<int, int>> _soldiers; #endregion #region Tulajdonságok public int CastleLives => _castleLives; public int TimeElapsed => _timeElapsed; public bool IsGameOver => _castleLives <= 0; // Kiszámoljuk, vehetünk-e még katonát: // Alap 2 + minden 3. ölés után 1 - amennyit már leraktunk public int AvailableSoldiers => (2 + (_enemiesKilled / 3)) - _deployedSoldiers; #endregion #region Események public event EventHandler? GameAdvanced; // Léptetéskor (idő, mozgás) public event EventHandler? GameOver; // Játék vége #endregion }</pre>	<pre>#region Konstruktor public InvasionGameModel(ITimer timer) { _timer = timer; _timer.Interval = GameTickSpeed; _timer.Elapsed += Timer_Elapsed; _random = new Random(); _enemies = new List<Tuple<int, int>>(); _soldiers = new List<Tuple<int, int>>(); } #endregion #region Publikus metódusok public void NewGame() { _castleLives = 3; _enemiesKilled = 0; _deployedSoldiers = 0; _timeElapsed = 0; _enemies.Clear(); _soldiers.Clear(); _timer.Start(); // Azonnali frissítés, hogy a UI alaphelyzetbe álljon GameAdvanced?.Invoke(this, EventArgs.Empty); } public void PlaceSoldier(int x, int y) { if (IsGameOver) return; // Csak a fal elé, üres helyre rakhatunk, ha van keretünk if (x < Rows - 1 && // Nem a falra !IsOccupied(x, y) && AvailableSoldiers > 0) { _soldiers.Add(new Tuple<int, int>(x, y)); _deployedSoldiers++; GameAdvanced?.Invoke(this, EventArgs.Empty); } }</pre>
---	--

<pre> // Segédfüggvény a nézetnek: megmondja, mi van az adott koordinátán public FieldType GetFieldValue(int x, int y) { if (x == Rows - 1) return FieldType.Wall; // Az utolsó sor a fal // Koordináta keresése a listákban var pos = new Tuple<int, int>(x, y); if (_soldiers.Contains(pos)) return FieldType.Soldier; if (_enemies.Contains(pos)) return FieldType.Enemy; return FieldType.Empty; } #endregion #region Privát logika private bool IsOccupied(int x, int y) { var pos = new Tuple<int, int>(x, y); return _soldiers.Contains(pos) _enemies.Contains(pos); } private void Timer_Elapsed(object? sender, EventArgs e) { _timeElapsed++; // 1. Új ellenség generálása (kb. 30% esély minden ütemben) if (_random.NextDouble() < 0.3) { int startY = _random.Next(Columns); // Csak akkor jön létre, ha a belépési pont üres if (!IsOccupied(0, startY)) { _enemies.Add(new Tuple<int, int>(0, startY)); } } // 2. Mozgatás MoveEnemies(); // 3. Ütközések és harc CheckCollisions(); GameAdvanced?.Invoke(this, EventArgs.Empty); // 4. Játék vége ellenőrzés if (IsGameOver) { _timer.Stop(); GameOver?.Invoke(this, EventArgs.Empty); } } private void MoveEnemies() { for (int i = 0; i < _enemies.Count; i++) { // Mindenki egyet lép lefelé (X koordináta növelése) var oldPos = _enemies[i]; _enemies[i] = new Tuple<int, int>(oldPos.Item1 + 1, oldPos.Item2); } } private void CheckCollisions() { List<Tuple<int, int>> deadEnemies = new List<Tuple<int, int>>(); List<Tuple<int, int>> deadSoldiers = new List<Tuple<int, int>>(); foreach (var enemy in _enemies) { // A) Elérte a falat? (A fal az utolsó sor: Rows - 1) if (enemy.Item1 == Rows - 1) { _castleLives--; deadEnemies.Add(enemy); continue; } // B) Ütközés katonával (ugyanazon a mezőn) if (_soldiers.Contains(enemy)) { deadEnemies.Add(enemy); deadSoldiers.Add(enemy); // A katona is meghal // Itt nem növeljük a _enemiesKilled-et, mert "kiesnek", nem mi öltük meg aktívan (opcionális értelmezés) } } } </pre>	<pre> // De a feladat szerint "minden harmadik megölt ellenség", így számolhatjuk ezt is: _enemiesKilled++; continue; } // C) Elhaladás katona mellett (szomszédos mezőn van katona) // Megnézzük a 4 szomszédot bool soldierNearby = false; int[] dx = { -1, 1, 0, 0 }; int[] dy = { 0, 0, -1, 1 }; for (int k = 0; k < 4; k++) { var neighbor = new Tuple<int, int>(enemy.Item1 + dx[k], enemy.Item2 + dy[k]); if (_soldiers.Contains(neighbor)) { soldierNearby = true; break; } } if (soldierNearby) { deadEnemies.Add(enemy); _enemiesKilled++; } // Törlések végrehajtása foreach (var e in deadEnemies) _enemies.Remove(e); foreach (var s in deadSoldiers) _soldiers.Remove(s); } ViewModel : Field public class InvasionField : ViewModelBase { private string _color = "White"; private string _text = ""; public int X { get; set; } public int Y { get; set; } public string Color { get => _color; set { if (_color != value) { _color = value; OnPropertyChanged(); } } } public string Text { get => _text; set { if (_text != value) { _text = value; OnPropertyChanged(); } } } // Parancs, ha rákattintanak (katona lerakása) public RelayCommand? ClickCommand { get; set; } } </pre>
--	--

<pre> ViewModel private readonly InvasionGameModel _model; // A rács mezői public ObservableCollection<InvasionField> Fields { get; private set; } // Státusz szövegek public string StatusText => \$"Élet: { _model.CastleLives } Idő: { _model.TimeElapsed } mp Katona: { _model.AvailableSoldiers } db"; public InvasionViewModel(InvasionGameModel model) { _model = model; // Események feliratkozása _model.GameAdvanced += (s, e) => UpdateView(); _model.GameOver += (s, e) => { UpdateView(); // Itt jelezhetnénk a végén (pl. szöveg átírása), de a státuszsorban // is látszik majd }; // Rács inicializálása Fields = new ObservableCollection<InvasionField>(); for (int i = 0; i < InvasionGameModel.Rows; i++) { for (int j = 0; j < InvasionGameModel.Columns; j++) { int x = i; int y = j; // Helyi másolat a lambda miatt Fields.Add(new InvasionField { X = x, Y = y, ClickCommand = new RelayCommand(() => _model.PlaceSoldier(x, y)) }); } } _model.NewGame(); UpdateView(); } private void UpdateView() { // Mivel az időzítő más szálon fut, vissza kell térnünk a UI szálra Dispatcher.UIThread.InvokeAsync(() => { // Státuszsor frissítése OnPropertyChanged(nameof(StatusText)); // Minden mező színének és szövegének frissítése a modell alapján foreach (var field in Fields) { var type = _model.GetFieldValue(field.X, field.Y); switch (type) { case Fieldtype.Wall: field.Color = "Gray"; field.Text = "FAL"; break; case Fieldtype.Enemy: field.Color = "Red"; field.Text = "O"; // Ellenség jele break; case Fieldtype.Soldier: field.Color = "Blue"; field.Text = "X"; // Katona jele break; default: field.Color = "White"; field.Text = ""; break; } } }); } </pre>	<pre> MainWindow <Window xmlns="https://github.com/avaloniaui" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml" xmlns:vm="using:mintaZh_Invazio_Avalonia.ViewModels" xmlns:d="http://schemas.microsoft.com/expression/blend/2008" xmlns:mc="http://schemas.openxmlformats.org/markup- compatibility/2006" xmlns:views="clr-namespace:mintaZh_Invazio_Avalonia.Views" mc:Ignorable="d" d:DesignWidth="400" d:DesignHeight="600" x:Class="mintaZh_Invazio_Avalonia.Views.MainWindow" x:DataType="vm:InvasionViewModel" Icon="/Assets/avalonia-logo.ico" Title="Invázio Játék" Width="450" Height="700"> <Grid RowDefinitions="Auto, *"> <Border Grid.Row="0" Background="LightGray" Padding="10"> <TextBlock Text="{Binding StatusText}" HorizontalAlignment="Center" FontSize="18" FontWeight="Bold"/> </Border> <ItemsControl Grid.Row="1" ItemsSource="{Binding Fields}" Margin="10"> <ItemsControl.ItemsPanel> <ItemsPanelTemplate> <UniformGrid Rows="12" Columns="6"/> </ItemsPanelTemplate> </ItemsControl.ItemsPanel> <ItemsControl.ItemTemplate> <DataTemplate> <Button Command="{Binding ClickCommand}" Background="{Binding Color}" HorizontalAlignment="Stretch" VerticalAlignment="Stretch" HorizontalContentAlignment="Center" VerticalContentAlignment="Center" BorderThickness="1" BorderBrush="Black" CornerRadius="0"> <TextBlock Text="{Binding Text}" FontSize="20" FontWeight="Bold" Foreground="Black"/> </Button> </DataTemplate> </ItemsControl.ItemTemplate> </ItemsControl> </Grid> </Window> App public override void Initialize() { AvaloniaXamlLoader.Load(this); } public override void OnFrameworkInitializationCompleted() { // Csak akkor indulunk el, ha asztali környezetben vagyunk if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop) { // 1. Modell létrehozása (a SudokuTimerInheritance osztályt // használva) var gameModel = new InvasionGameModel(new SudokuTimerInheritance()); // 2. ViewModel létrehozása var viewModel = new InvasionViewModel(gameModel); // 3. Ablak létrehozása és összekötés desktop.MainWindow = new MainWindow { DataContext = viewModel }; } base.OnFrameworkInitializationCompleted(); } </pre>
---	---

Feladat: Huszárpárba

Készítsünk egy *Avalonia UI* grafikus felületű, *MVVM* architektúrájú cross-platform alkalmazást *C#* nyelven, amelyben két játékos lovakkal foglal el mezőket egy táblán. A játékosok felváltva lépkedhetnek „L” alakban a lovaikkal, minden lépéssel elfoglalva egy mezőt. Elfoglalt mezőkre nem lehet újra lépni. Ha az egyik játékosnak nincsen legális lépése, a másik játékos nyer.

Részfeladatok:

2. A program indításakor generáljunk egy 8 x 8-as játéktáblát és rakjuk az egyik játékos lovát a jobb alsó, a másik játékos lovát a bal felső sarokba. Az alap háttérszíne minden mezőnek szürke legyen. Ha az első játékos rálép egy mezőre, azt színezzük fehérre, ha a második játékos, akkor feketére. A lovak kezdeti mezőjét is színezzük ki ez alapján. Érvénytelen lépést (már színezett mezőre lépést) ne engedélyezzen a játék. Legyen bármikor lehetőség új játék indítására.

Az alkalmazást elegendő asztali alkalmazásként elkészíteni.

<pre>GameModel public class PlayerTurnChangedEventArgs : EventArgs { private readonly Player _currentPlayer; public PlayerTurnChangedEventArgs(Player p) { _currentPlayer = p; } public Player CurrentPlayer => _currentPlayer; } public class GameOverEventArgs : EventArgs { private readonly Player _winner; public GameOverEventArgs(Player p) { _winner = p; } public Player Winner => _winner; } public enum Player { None, Player1, Player2 } private Player _currentPlayer = Player.Player1; private Player[,] _board; public Tuple<int, int> CurrentPlayer1Position { get; set; } public Tuple<int, int> CurrentPlayer2Position { get; set; } public Player CurrentPlayer { get => _currentPlayer; set { if (_currentPlayer != value) { _currentPlayer = value; } } } public Player[,] Board => _board; public event EventHandler<PlayerTurnChangedEventArgs>? PlayerTurnChanged; public event EventHandler<GameOverEventArgs>? GameOver; public GameModel() { _board = new Player[8, 8]; _board[_board.GetLength(0) - 1, _board.GetLength(1) - 1] = Player.Player1; _board[0, 0] = Player.Player2; CurrentPlayer1Position = Tuple.Create(_board.GetLength(0) - 1, _board.GetLength(1) - 1); CurrentPlayer2Position = Tuple.Create(0, 0); }</pre>	<pre> } public void NewGame() { _board = new Player[8, 8]; _board[_board.GetLength(0) - 1, _board.GetLength(1) - 1] = Player.Player1; _board[0, 0] = Player.Player2; CurrentPlayer = Player.Player1; } public void MakeMove(int fromRow, int fromCol, int toRow, int toCol) { if (!IsValidMove(fromRow, fromCol, toRow, toCol)) { _board[toRow, toCol] = _board[fromRow, fromCol]; if (CurrentPlayer == Player.Player1) { CurrentPlayer1Position = Tuple.Create(toRow, toCol); } else { CurrentPlayer2Position = Tuple.Create(toRow, toCol); } CurrentPlayer = CurrentPlayer == Player.Player1 ? Player.Player2 : Player.Player1; PlayerTurnChanged?.Invoke(this, new PlayerTurnChangedEventArgs(CurrentPlayer)); IsGameOver(CurrentPlayer); } } public void IsGameOver(Player player) { bool hasValidMove = false; // A huszár 8 lehetséges ugrása int[] dx = { -2, -2, -1, -1, 1, 1, 2, 2 }; int[] dy = { -1, 1, -2, 2, -2, 2, -1, 1 }; int currentRow, currentCol; // Megnézzük, kinek a pozícióját kell vizsgálni if (player == Player.Player1) { currentRow = CurrentPlayer1Position.Item1; currentCol = CurrentPlayer1Position.Item2; } else { currentRow = CurrentPlayer2Position.Item1; currentCol = CurrentPlayer2Position.Item2; } }</pre>
--	--

<pre> // Végigpróbáljuk a lehetséges lépéseket for (int i = 0; i < 8; i++) // Ha átírod L-alakr, itt i < 8 lesz { int newRow = currentRow + dx[i]; int newCol = currentCol + dy[i]; if (IsValidMove(currentRow, currentCol, newRow, newCol)) { hasValidMove = true; break; // Találtunk lépést, nem kell tovább keresni } } // HIBA JAVÍTVA: Akkor van vége, ha NINCS érvényes lépés if (!hasValidMove) { // Ha a "player" nem tud lépni, akkor a MÁSIK játékos nyert Player winner = (player == Player.Player1) ? Player.Player2 : Player.Player1; GameOver?.Invoke(this, new GameOverEventArgs(winner)); } } private bool IsValidMove(int fromRow, int fromCol, int toRow, int toCol) { if (toRow < 0 toRow >= _board.GetLength(0) toCol < 0 toCol >= _board.GetLength(1)) return false; if (_board[toRow, toCol] != Player.None) return false; int rowDiff = Math.Abs(toRow - fromRow); int colDiff = Math.Abs(toCol - fromCol); // L-alak: egyik irányba 2, másikba 1 lépés return (rowDiff == 2 && colDiff == 1) (rowDiff == 1 && colDiff == 2); } </pre>	<pre> ViewModel : Field public class FieldViewModel : ViewModelBase { private readonly GameModel _model; private string _text = ""; public string Text { get => _text; set { if (_text != value) { _text = value; OnPropertyChanged(); } } } public int Row { get; set; } public int Col { get; set; } public FieldViewModel(GameModel model, int row, int col) { _model = model; Row = row; Col = col; } public string BackGround => _model.Board[Row, Col] switch { GameModel.Player.Player1 => "White", GameModel.Player.Player2 => "Black", _ => "Gray" }; public RelayCommand? ClickCommand { get; set; } public void Refresh() { OnPropertyChanged(nameof(BackGround)); } } </pre>
---	--

<pre> ViewModel private readonly GameModel _model; public RelayCommand NewGameCommand { get; private set; } public ObservableCollection<FieldViewModel> Fields { get; } public event EventHandler? NewGame; public MainViewModel(GameModel model) { _model = model; Fields = new ObservableCollection<FieldViewModel>(); for (int row = 0; row < 8; row++) { for (int col = 0; col < 8; col++) { var fieldVM = new FieldViewModel(_model, row, col); fieldVM.ClickCommand = new RelayCommand(() => OnFieldClicked(fieldVM)); Fields.Add(fieldVM); } } NewGameCommand = new RelayCommand(OnNewGame); _model.NewGame(); } private void OnFieldClicked(FieldViewModel fieldVM) { _model.MakeMove(_model.CurrentPlayer == GameModel.Player.Player1 ? _model.CurrentPlayer1Position.Item1 : _model.CurrentPlayer2Position.Item1, _model.CurrentPlayer == GameModel.Player.Player1 ? _model.CurrentPlayer1Position.Item2 : _model.CurrentPlayer2Position.Item2, fieldVM.Row, fieldVM.Col); foreach (var field in Fields) { field.Refresh(); } } private void OnNewGame() { _model.NewGame(); foreach (var field in Fields) { field.Refresh(); } NewGame?.Invoke(this, EventArgs.Empty); } </pre>	<pre> MainWindow <Grid RowDefinitions="Auto,*" Background="LightBlue"> <Button Grid.Row="0" Content="New Game" Command="{Binding NewGameCommand}" Margin="10" HorizontalAlignment="Left"/> <ItemsControl Grid.Row="1" ItemsSource="{Binding Fields}"> <ItemsControl.ItemsPanel> <ItemsPanelTemplate> <UniformGrid Rows="8" Columns="8"/> </ItemsPanelTemplate> </ItemsControl.ItemsPanel> <ItemsControl.ItemTemplate> <DataTemplate> <Button Command="{Binding ClickCommand}" Background="{Binding BackGround}"> <TextBlock Text="{Binding Text}" FontSize="20" FontWeight="Bold" Foreground="Black"/> </Button> </DataTemplate> </ItemsControl.ItemTemplate> </ItemsControl> </Grid> APP public override void Initialize() { AvaloniaXamlLoader.Load(this); } public override void OnFrameworkInitializationCompleted() { // Line below is needed to remove Avalonia data validation. // Without this line you will get duplicate validations from both Avalonia and CT BindingPlugins.DataValidators.RemoveAt(0); if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop) { var gameModel = new GameModel(); gameModel.GameOver += Model_GameOver; var viewModel = new MainViewModel(gameModel); desktop.MainWindow = new MainWindow { DataContext = viewModel }; } base.OnFrameworkInitializationCompleted(); } private void Model_GameOver(object? sender, GameModel.GameOverEventArgs e) { Dispatcher.UIThread.InvokeAsync(() => { MessageBoxManager.GetMessageBoxStandard("Játék vége", \$"Játék vége!\n\nGYŐZTES: {e.Winner}\n\nGratulálunk!", ButtonEnum.Ok, Icon.Info).ShowAsync(); }); } </pre>
---	---